

Agentic Customer 360 — Proactive Intervention Desk

Mid-Term Submission — Research Log, Preliminary Architecture & Approach

Inter IIT Tech Meet 15.0 · Prepathon · Natural Language Processing

Submitted by: **Naman Goyal**

Date: **15 September 2026**

The MAS topology, memory scoping and trigger design in Part C were settled before any code existed, and Part B records what informed each one. AI tooling was used to implement and debug that design and to calibrate thresholds against the practice scenarios — not to choose it. Where the design did change during the build it changed in response to what the data showed, and those changes are recorded in the evaluation write-up.

Part A — Findings & Approach Report

What I understand about the domain

The challenge here is not throughput or data volume — the streams provided (~500 events per customer over ~2.5 months) are small. It is temporal judgement: withholding a decision while a signal is still weak, then committing within a narrow window once it is strong enough — without being fooled by isolated anomalous events (“red herrings”) that resemble a different life event or risk category.

A second, quieter difficulty sits underneath it. Each event carries two timestamps: when it happened, and when the bank's systems learned of it. Any claim to have inferred something “early” is only meaningful if the system provably could not have seen the future — including data that had occurred but had not yet arrived. That is a stream-processing problem rather than a retrieval one, and it shapes the memory design more than anything else.

So this is an agent-coordination and memory-scoping problem more than a data-engineering one — consistent with the problem statement's own framing (§1.3: “this is a genuinely unfamiliar problem space... reading how others have approached these problems is a load-bearing part of the work”).

Approach I am following

- **Multi-layer MAS.** A parallel (“swarm”) perception layer of specialised agents (Usage, Transaction, Support, Life-Signal/KYC) feeding a single Synthesis agent, followed by a Critique-Refiner pass before any action is finalised.
- **Per-customer shared state board** rather than siloed agent memory or agent-to-agent messaging — the single surface each perception agent writes structured findings to and the correlation layer reads from.
- **Three-tier memory.** Working (the tick being handled), episodic (per-customer event history in a queryable store, strictly time-filtered), semantic (shared policy and pattern knowledge in a vector store).
- **A hard, code-level guardrail** requiring at least two independent source_systems to corroborate before any escalation, offer or fraud-hold action — aimed directly at the red-herring pattern present in all three provided scenarios.
- **Model routing.** A fast model for high-volume perception, a stronger reasoning model for Synthesis and Action — a deliberate cost/quality tradeoff, not one model used everywhere. Every model call has a deterministic fallback, so the system runs end to end with no API access at all.

Progress at mid-term

The architecture in Part C is fixed, and every component in it is built and integrated — ingestion through to the inferred-events writer — with a local scoring harness and a terminal view running over the same pipeline the graded run uses. Remaining work before the end-term submission: calibration reporting, one recorded run against the live models, and extending the research log below.

Part B — Research Log

Not a bibliography — every row is here because it changed a decision that is visible in the system.

Source	Type	Key takeaway	Decision it informed
Akidau, “Streaming 101 / 102: The world beyond batch” (O'Reilly Radar)	Engineering write-up	Event time is when something occurred; processing time is when the system observed it. The skew between them is “not only non-zero, but often a highly variable function” of the source and the pipeline, so correctness has to be stated in event time even though records physically arrive in processing-time order. Watermarks estimate how far event time has advanced; late data is the normal case, not an error.	The two-timestamp design. <code>release_time = max(ingestion_time, event_time)</code> is our watermark: an event cannot be released before it happened, so the no-leakage rule holds by construction rather than by test. It also gives the visibility filter its second clause, <code>release_time ≤ as_of</code> , which stops a late-arriving event appearing to have been known before it reached the bank.
Park et al., “Generative Agents: Interactive Simulacra of Human Behavior” (2023), §4	Research paper	Agents retrieve from a memory stream scored on recency, importance and relevance. Separately, reflection synthesises raw observations into higher-level conclusions the agent then reasons over instead of re-deriving them. The value is in promoting observations into stable conclusions, not in storing more.	The working / episodic / semantic split, and the state board specifically: perception publishes structured findings and Synthesis reads only those, never the event stream — that is the reflection layer here. It also justifies carrying a committed inference forward rather than recomputing it each day.
Packer et al., “MemGPT: Towards LLMs as Operating Systems” (2023), and the Letta docs on tiered memory	Research paper / engineering docs	Hierarchical memory borrowed from operating systems: a small in-context tier plus a larger external store, with movement between them an explicit operation rather than an invisible mechanism. The argument is less about capacity than about making retrieval addressable, and therefore constrainable.	Episodic memory as an explicitly queried SQLite store rather than context stuffing — and, because every read is an explicit call, <code>as_of</code> could be made a mandatory positional argument with no default. A caller physically cannot query memory without stating what “now” is.
LangChain — “LangGraph: Multi-Agent Workflows” and the LangGraph multi-agent concepts docs	Official blog / docs	Compares collaboration, supervisor and hierarchical topologies. What separates them is whether agents see each other's work in progress: under collaboration “all the work either of them do is visible to the other”, whereas a supervisor gives each agent “their own independent scratchpads” with only final outputs reaching shared state.	A swarm at Layer 1 writing to a shared board for outputs only, with no agent-to-agent messaging — made concrete by constructing each agent's context without a reference to the board. If Support could see Usage's finding, one piece of evidence would be counted twice while appearing to be two independent witnesses, and corroboration is the guardrail's whole basis.
Anthropic — “Building Effective Agents”	Engineering blog	Recommends “finding the simplest solution possible, and only increasing complexity when needed”, and warns that “agentic systems often trade latency and cost for better task performance”. Its evaluator-optimizer pattern — one call generates, a second evaluates in a loop — is the shape that fits a bounded quality gate.	Rejecting multi-agent debate: with one customer and six permitted actions it costs latency and free-tier quota without buying accuracy, so one adversarial reviewer with a single bounded retry is enough. It also informed keeping the confidence arithmetic and the guardrail in code — only genuine judgement is asked of a model.

Source	Type	Key takeaway	Decision it informed
ChromaDB documentation — collections, upsert, custom embedding functions	Official docs	upsert writes or updates by document id, so re-seeding a collection is idempotent and editing one document re-embeds only that document. Embedding functions are pluggable, and the default downloads a model on first use.	Content-hashed incremental upsert, so the policy store stays live without a full re-index. The first-use download is why there is a dependency-free fallback embedder: a sandboxed evaluation run with no network still completes end to end.

Part C — System Architecture

1. Problem framing, grounded in the provided scenarios

Three scenarios were provided as practice data, each one customer's event timeline (~500 events across history_seed.jsonl and live_stream.jsonl), graded at fixed checkpoints against a hidden ground truth:

Scenario	Customer	True narrative	Checkpoints	Final action	Red herring(s)
1 — Medical hardship	Marcus Vance (CUST_00088)	ER visit → income drop (benefits) → hospital bill → savings drawn down → support ticket	low → medium → high	support_intervention (escalated)	Tuition transfer; resort refund
2 — New child	Priya Sharma (CUST_00105)	Income dip → baby purchases → daycare standing instruction → KYC dependents change → search intent	low → high (no medium)	personalized_offer (escalated)	Baby monitor purchase
3 — Churn risk	David Chen (CUST_00184)	Denied complaint → usage drop → cancelled standing instructions + transfer out → zero card use → salary swept out	low → high → high (2nd action)	relationship_manage_r_escalation, later proactive_retention_outreach	Tax refund deposit

The common pattern across all three: every red herring is a single isolated event, uncorroborated by any other source_system in the same window, while every genuine narrative shows up across several. That observation is what the guardrail in §6 is built on.

2. The environment the agents live in

Ambient agents need somewhere to be ambient. Here that is the provided event stream, replayed as a live environment rather than read as a file: history_seed.jsonl is loaded once as backstory, and live_stream.jsonl is released one event at a time in the order a real system would have received them. Across the three practice scenarios that is 1,367 events from nine source systems over a 74-day window (1 February to 15 April 2026).

Release order is ingestion_time; judgement is event_time. An event is released at max(ingestion_time, event_time), so it can never surface before it happened. Two of the 298 live events arrive late — after the day they occurred — and both are ground-truth signal events, which is why visibility is filtered on release_time as well as event_time.

Three trigger types drive the agents, and the system uses all three:

- **Event-based.** An event arrives and is handed straight to the one agent that owns its source system. This is the synchronous path, and it is why a cancellation click is visible the moment it lands.
- **Time-based.** A daily 00:00Z boundary wakes every agent whether or not anything arrived. This is how absence is detected — a salary that did not land, logins that stopped — none of which produces an event of its own.
- **Agent-dependent.** Synthesis fires only once two or more distinct perception agents have flagged something in the same window, so correlation is attempted when there is something to correlate rather than on every raw event.

The loop closes back into the environment. Each daily cycle writes a decision row to `inferred_events.json` and commits it to episodic memory, and the next day's cycle reads that decision back as the belief it starts from. Agents read from the stream and act into a record the next tick can see — which is what makes an inference persist rather than be recomputed from nothing each day.

3. Agent roster

Drawn in full in Appendices A and B.

Component	Layer	Reads	Trigger
Usage Agent	Perception (swarm)	web_app_events	Event-based and daily time-based
Transaction Agent	Perception (swarm)	card_payments, core_banking_ledger, instant_payments, ach_wire, trading_brokerage	Event-based and daily time-based
Support Agent	Perception (swarm)	support_logs	Event-based
Life-Signal Agent	Perception (swarm)	loan_kyc, social_signal_consented (consent-gated)	Event-based
Synthesis Agent	Correlation	The state board — structured findings only, never raw events	Agent-dependent: ≥ 2 distinct perception agents flagged in the window
Action Proposer	Decision	Synthesis output, retrieved bank policy, prior decisions	Downstream of Synthesis
Guardrail	Safety (code, not an agent)	The proposed action and its corroboration	Every proposal, before the Critique Agent
Critique Agent	Quality gate	The proposal, the diagnosis and the guardrail verdict	Only when an action other than <code>no_action</code> is proposed
HITL stub	Human checkpoint	The final action and the full reasoning trace	Any action $\neq$ <code>no_action</code>

4. MAS topology, by stage

Stage	Pattern	Justification
Signal gathering (Layer 1)	Swarm — parallel, independent	Four perception agents read disjoint <code>source_systems</code> with no interdependency, so they run in parallel at no cost to accuracy. Each is constructed without access to the state board, which is what makes their findings independent rather than merely concurrent.
Correlation (Layer 2)	Handoff to a single orchestrator	Synthesis gains nothing from running in parallel with itself; it needs the swarm's combined output, so it sits downstream of Layer 1 on an agent-dependent trigger.
Action quality gate (Layer 3)	Critique-Refiner	Targets the false-positive risk that is explicitly graded in all three scenarios. One bounded retry, not an open loop: on a free tier an unbounded critique exhausts quota early in a long replay and leaves the remainder running with no model at all.
Debate	Considered and rejected	Debate earns its place where genuine disagreement between agents needs surfacing rather than averaging away. Here that disagreement is already surfaced and settled by the affinity and hysteresis rule in §5, at no latency cost. A wider action set, or several customers in scope at once, would change the answer.

5. Memory architecture

Tier	Scope	Storage	Contents
Working	One in-flight decision	In-process	The tick currently being handled
Episodic	Per customer, persistent	SQLite, keyed by customer_id	Every event, finding and committed decision, timestamped; read only through a query carrying a mandatory as_of, filtered on event_time ≤ as_of AND release_time ≤ as_of
Semantic	Cross-customer, shared	ChromaDB	Narrative pattern descriptions per inferred_state, and bank policy text used to ground the proposed action and its subtype

As the problem statement requires, an inferred life-event state persists across checkpoints rather than being recomputed — it must still be available, and correctly weighted, weeks later.

Rolling statistics — login frequency, spend baselines — are maintained against the stream as events arrive rather than recomputed at query time, and the semantic store is updated by content-hashed upsert rather than a full re-index, so editing one policy re-embeds one document.

6. Guardrails, traceability and HITL

- **The hard rule is code-level, not prompt-level.** No compliance_fraud_hold, relationship_manager_escalation or personalized_offer may fire unless at least two independent source_systems corroborate the finding within the window, across at least two distinct events. Every check is arithmetic and set membership; no model is consulted, so nothing can be talked out of its answer.
- **The bar matches the cost of being wrong.** support_intervention and proactive_retention_outreach are deliberately unguarded. A support call to a customer who did not need one costs almost nothing; a frozen account, a mistimed sales offer or an unnecessary escalation are real harms. One threshold for everything would either block the cheap interventions for no benefit or set the bar too low for the expensive ones.
- **Disagreement is resolved by a stated rule, never silently.** Perception agents routinely support different states from the same window. Candidate states are scored by weighted signal affinity, and an established belief is displaced only if a challenger beats it by a margin that scales with how well-evidenced that belief was. When the top two candidates fall within 25% of each other the tie goes to the Synthesis model to adjudicate; with no model available the deterministic winner stands.
- **The guardrail runs before the critique pass,** not after. The cheap deterministic check that can definitively reject should precede the expensive judgement call that might not.
- **Any action other than no_action defaults to escalated,** matching the pattern in all three scenarios' ground truth. A rejection by a reviewer turns the action into no_action in the output rather than merely relabelling it.
- **Traceability:** one structured trace record per checkpoint carrying the findings, the affinity scores, the guardrail verdict, the critique verdict and the HITL routing, tagged with customer_id and as_of.
- **Explainability is enforced structurally:** an output row proposing an action cannot be constructed at all unless its notes cite at least one event id. A code check, not an instruction in a prompt.
- **PII is tokenised rather than deleted** before any prompt or log line, so that a self-transfer to another bank still reads as a self-transfer once the name is masked.

7. Design questions, and where they landed

Question	Resolution
Episodic-memory relevance and decay (§4.1): how far back should Synthesis look before an old flag is stale, and when is a past memory relevant rather than noise?	A 30-day correlation window for findings, chosen against the spacing of the practice narratives; a 60-day confidence decay for a belief with no supporting evidence at all, set longer than any gap inside a graded window so it exists for the hidden set rather than being fitted to this one; and a monotonic rule so an established belief does not drop a

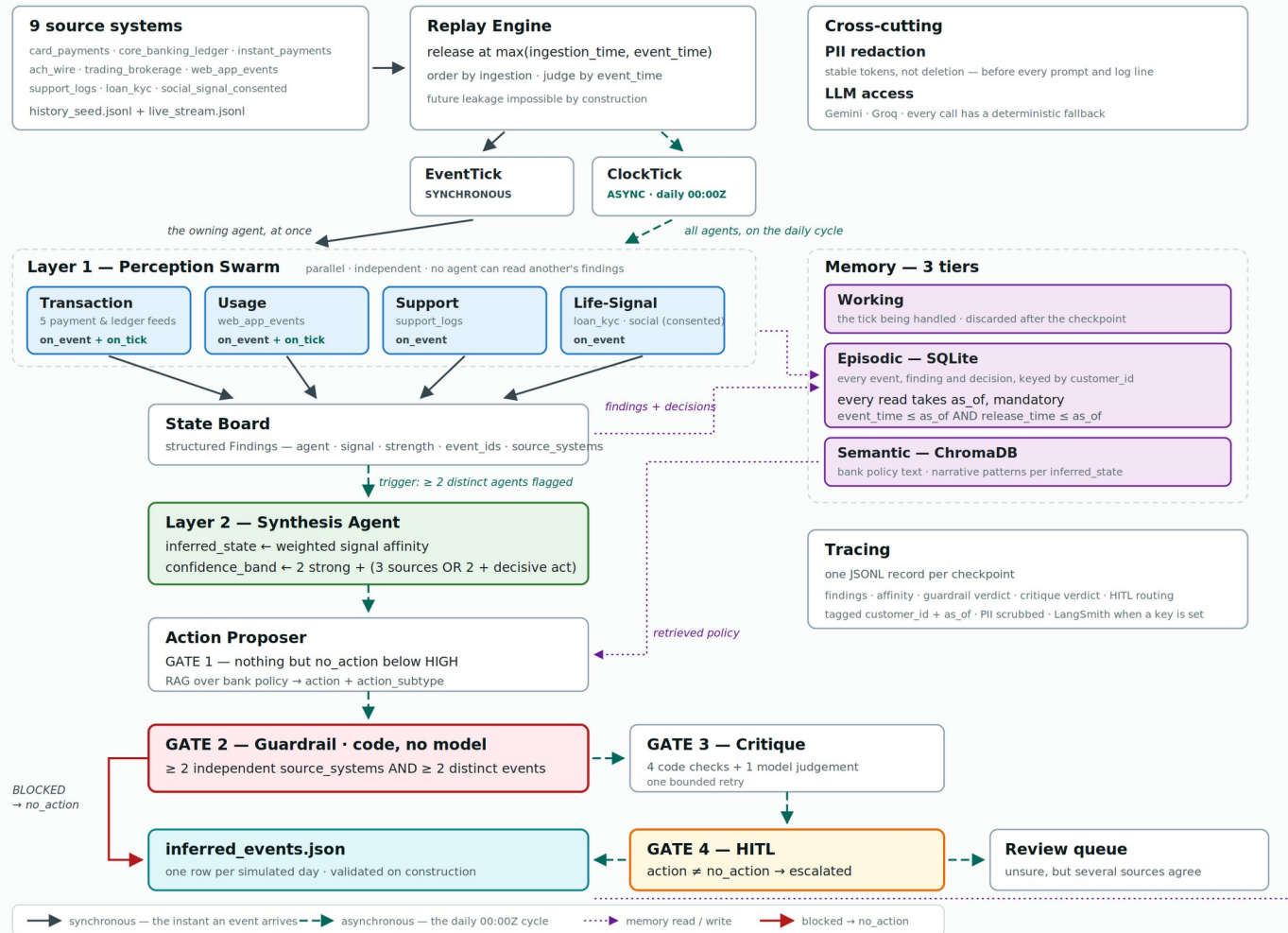
Question	Resolution
	band because one quiet fortnight thinned the window. Relevance is handled separately from recency: confidence and corroboration are computed only over findings that actually support the state being claimed, so an unrelated event inside the window cannot strengthen a narrative it says nothing about.
Memory bleed-through between customers (§4.1): how is one customer's history kept out of another's inference?	The store is keyed by customer_id and every read carries it, so isolation is a property of the query rather than a convention. At this scale each replay runs against its own store; the same predicate is what would scope a shared one.
Precise confidence-band thresholds — what quantifies low, medium and high?	Two distinct strong signals AND either three independent source systems, or two source systems plus one signal that is decisive — meaning the customer did something deliberate (filed a change, clicked cancel, moved money to their own account elsewhere) rather than something we merely measured about them. These were calibrated against the eight graded practice checkpoints, so scoring well on those shows the calibration was applied, not that it generalises.
Escalation for ambiguity rather than cost (§6.3): should low confidence, or an unresolved disagreement, reach a human even when no action is proposed?	Yes, through a second channel. The confidence gate turns low confidence into no_action, which is auto-approved — right for the graded field, but it meant the system was silent exactly where it was least sure. A review queue now raises a checkpoint when several independent source systems agree but not strongly enough to act, or when the top two candidate states score within 25% of each other. It is written beside the graded file and never alters it, because hitl_status is itself graded. It raises on change rather than persistence: a first version that flagged every qualifying day produced 38 items across scenario_01's 74 checkpoints, which is alert fatigue rather than oversight. It now yields 2, 4 and 1.
Whether to build a local scoring harness for the brownie-point credit in §8.1	Built. It scores the inferred-events output against each scenario's ground_truth.json on state, confidence, action, subtype and HITL status. It also checks lead time against the ideal action date, and that no action fires inside a red-herring window.

Appendix A — System Flow

Every component, and how work passes between them. Solid lines are synchronous, taken the instant an event arrives; dashed lines are asynchronous, on the daily 00:00Z cycle. Full resolution: [docs/architecture.svg](#)

Agentic Customer 360 — Proactive Intervention Desk

System flow · diagram 1 of 2 — each agent's tools, and the data source each tool reads from or writes to, are in the Agent & Tool Detail diagram



Appendix B — Agent & Tool Detail

Each agent's toolset, and the data source every tool reads from or writes to. Full resolution: docs/architecture_agents.svg

Agent & Tool Detail

Diagram 2 of 2 · every agent, the tools it can call, and the data source each tool reads from or writes to. Flow and handoffs are in the System Flow diagram.

COMPONENT	READS FROM	TOOLS → DATA SOURCE EACH ONE TOUCHES	PROMPT	EMITS / RETURNS
LAYER 1 — PERCEPTION SWARM · parallel, independent, disjoint source systems				
Transaction Agent on_event + on_tick agents/transaction.py	card_payments core_banking_ledger instant_payments ach_wire trading_brokerage	memory.events_as_of(as_of, ...) → episodic (SQLite) memory.daily_rate(as_of, ...) → episodic (SQLite) memory.baseline_daily_rate(as_of, ...) → episodic (SQLite) redactor.counterparty_is_customer → the event payload	none Every detection here is arithmetic, so no model is consulted at all.	income_disruption · income_replacement major_medical_expense · healthcare_spend savings_drawdown · large_outbound_transfer self_transfer_external · salary_swept_out standing_instruction_stopped · card_spend_collapse new_recurring_commitment · baby_retail_spend large_inbound_deposit · unusual_refund (14)
Usage Agent on_event + on_tick agents/usage.py	web_app_events	memory.daily_rate(as_of, ...) → episodic (SQLite) memory.baseline_daily_rate(as_of, ...) → episodic (SQLite) redactor.scrub → search_text in the event payload llm.complete_json → prompts.SEARCH_INTENT	SEARCH_INTENT → {intent, confidence, reason} fallback: INTENT_KEYWORDS, longest matched phrase wins	cancellation_feature_used engagement_drop session_length_collapse search_intent
Support Agent on_event agents/support.py	support_logs	payload.resolution_status → support_logs a structural read — needs no model redactor.scrub → ticket raw_text llm.complete_json → prompts.TICKET_THEME	TICKET_THEME → {theme, customer_distress, reason} told whether the bank or the customer wrote the text	complaint_denied complaint_open hardship_contact reassurance_contact
Life-Signal Agent on_event agents/life_signal.py	loan_kyc social_signal_consented	payload.consent_flag gate → social_signal_consented a record with no consent flag is never read, whatever it contains redactor.scrub → post raw_text llm.complete_json → prompts.SOCIAL_LIFE_EVENT	SOCIAL_LIFE_EVENT → {life_event, confidence, reason} fallback: SOCIAL_KEYWORDS	dependents_increase marital_status_change address_change loan_application social_life_event
LAYER 2 & 3 — CORRELATION AND DECISION · reached only on the daily cycle, never on the event path				
Synthesis Agent synthesis.py	the state board — findings only, never raw events	board.corroboration(as_of) → episodic findings board.current_state(as_of) → episodic decisions llm.complete_json → prompts.SYNTHESIS	SYNTHESIS → {inferred_state, rationale} called only when the top two states score within 25%	inferred_state · confidence_band rationale · event_ids · affinity strong_signals · carried_forward
Action Proposer action.py	the Synthesis result (never raw events)	semantic.retrieve_policies(query, k=4) → ChromaDB memory.decisions_as_of(as_of) → episodic decisions llm.complete_json → prompts.PROPOSER gate 1 first: nothing but no_action below high confidence	PROPOSER → {action, action_subtype, rationale} fallback: the highest-ranked eligible policy's own action	action · action_subtype policy_ids · policy_text gate_reason · decided_by
Guardrail guardrail.py · code	the proposal and its Corroboration record	NONE — set membership and len(), nothing else no model is consulted, so nothing can be talked out of its answer ≥ 2 independent source_systems AND ≥ 2 distinct events	none, by design	passed · reason · checks independent_sources · event_ids
Critique Agent critique.py	the proposal, the diagnosis and the guardrail verdict	4 checks in code: corroborated · policy-backed · no contradiction · ≥ 2 distinct events llm.complete_json → prompts.CRITIC proportionality is the only question a model is asked	CRITIC → {verdict, reason} one bounded retry (max 1); fallback: accept	verdict: accept downgrade reject reason · checks revised_action · revised_subtype
HITL checkpoint critique.py · HITLStub	the final action and its full reasoning trace	a CLI prompt to a human — approve / reject / modify a rejection turns the action into no_action in the output, rather than recording a rejection while still emitting the action	none	hitl_status · action action_subtype · note
Review queue review.py	the Synthesis result and the final proposal	NONE — comparison and set arithmetic, no model written to review_queue.json, alongside the graded file, never into it raises on a change in the picture, not on its persistence	none, by design	reason: weak_but_corroborated contested inferred_state · confidence_band source_systems · citations

Every model call above has a deterministic fallback that the test suite exercises: C360_OFFLINE=1 runs the whole system, end to end, with no API access.

Every prompt named here lives in one file, src/c360/prompts.py. Every tool's as_of argument is mandatory and has no default, so a caller cannot omit the time filter.

Findings are structured records (agent · signal · strength · event_ids · source_systems), never prose — the guardrail counts source_systems and the output cites event_ids.